

COMPLETE MASTER ROADMAP

DeepSeek Engineering Blog Series

10 Phases · 50+ Articles · Complete Resources
From Transformer Internals to DeepSeek-V4

Phase 1: LLM Foundations

Phase 2: KV Cache

Phase 3: MLA

Phase 4: Positional Enc.

Phase 5: MoE

Phase 6: MTP

Phase 7: Quantization

Phase 8: V2/V3 Design

Phase 9: R1 & Reasoning

Phase 10: Future Systems

Covering: Transformers · Attention · KV Cache · MLA · MQA · GQA · RoPE · MoE
MTP · Quantization · FP8 · GRPO · PPO · Chain-of-Thought · V2 · V3 · R1 · V4

Contents

1	How to Use This Roadmap	3
1.1	The Pyramid Writing Model — Applied to Every Article	3
1.2	Difficulty Legend	3
1.3	Pre-Writing Checklist (Every Article)	3
2	Full Series Map — All 50+ Articles	4
3	Phase 1 — LLM Foundations	5
3.1	1.1 — Introduction to DeepSeek Architecture	5
3.2	1.2 — How Tokens Flow Through an LLM	5
3.3	1.3 — Attention Mechanism Explained	6
3.4	1.4 — Self-Attention From Scratch	6
3.5	1.5 — Causal Attention and Autoregressive Generation	6
3.6	1.6 — Multi-Head Attention Internals	7
3.7	1.7 — Multi-Head Attention Implementation in Python	7
4	Phase 2 — KV Cache & Efficient Attention	9
4.1	2.1 — KV Cache Internals and Memory Bottlenecks	9
4.2	2.2 — Multi-Query Attention (MQA) Explained	9
4.3	2.3 — Grouped Query Attention (GQA) Deep Dive	9
4.4	2.4 — Why Traditional Attention Scaling Breaks	10
5	Phase 3 — DeepSeek MLA (Major Innovation)	11
5.1	3.1 — Multi-Head Latent Attention (MLA) Explained	11
5.2	3.2 — MLA Implementation From Scratch	11
5.3	3.3 — MLA vs MQA vs GQA Comparison	12
5.4	3.4 — How DeepSeek Reduced KV Cache Memory	12
5.5	3.5 — DeepSeek MLA + RoPE Architecture	12
6	Phase 4 — Positional Encoding Evolution	14
6.1	4.1 — Integer Positional Encoding	14
6.2	4.2 — Binary Positional Encoding	14
6.3	4.3 — Sinusoidal Positional Encoding Explained	14
6.4	4.4 — Rotary Positional Encoding (RoPE) Visual Guide	14
6.5	4.5 — Why RoPE Became Standard in Modern LLMs	15
7	Phase 5 — Mixture of Experts (MoE)	16
7.1	5.1 — Introduction to Mixture of Experts	16
7.2	5.2 — MoE Routing and Sparse Activation	16
7.3	5.3 — Visualizing Expert Selection	16
7.4	5.4 — Auxiliary Loss and Load Balancing	17
7.5	5.5 — Capacity Factor in MoE Systems	17
7.6	5.6 — How DeepSeek Rewrote MoE Architecture	17
7.7	5.7 — Coding MoE From Scratch in Python	18
8	Phase 6 — Multi-Token Prediction (MTP)	19
8.1	6.1 — Why Next-Token Prediction is Limited	19
8.2	6.2 — Introduction to Multi-Token Prediction	19
8.3	6.3 — How DeepSeek Improved MTP	19
8.4	6.4 — MTP Training Architecture	20
8.5	6.5 — MTP Implementation From Scratch	20

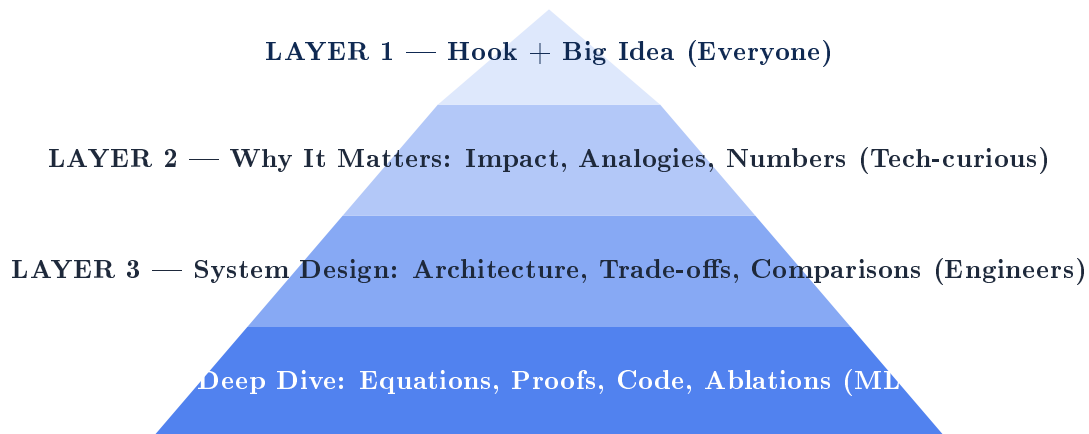
8.6	6.6 — Speculative Decoding and Fast Generation	20
9	Phase 7 — Quantization & Inference Optimization	21
9.1	7.1 — Introduction to LLM Quantization	21
9.2	7.2 — Mixed Precision Training Explained	21
9.3	7.3 — Fine-Grained Quantization in DeepSeek	21
9.4	7.4 — Accumulation Precision and Stability	22
9.5	7.5 — Online Quantization Techniques	22
9.6	7.6 — FP8 Training and Inference Scaling	22
10	Phase 8 — DeepSeek V2/V3 System Design	24
10.1	8.1 — DeepSeek V2 Architecture Breakdown	24
10.2	8.2 — DeepSeek V3 Architecture Explained	24
10.3	8.3 — Auxiliary-Loss-Free Load Balancing	24
10.4	8.4 — Sparse Compute Economics	25
10.5	8.5 — Distributed Training for MoE Models	25
10.6	8.6 — Expert Parallelism at Scale	25
10.7	8.7 — Communication Bottlenecks in LLM Training	26
10.8	8.8 — GPU Memory Optimization Strategies	26
11	Phase 9 — DeepSeek R1 & Reasoning	27
11.1	9.1 — DeepSeek R1 Architecture	27
11.2	9.2 — Reinforcement Learning for Reasoning Models	27
11.3	9.3 — GRPO vs PPO Explained	27
11.4	9.4 — Emergent Chain-of-Thought Reasoning	28
11.5	9.5 — Self-Verification in Reasoning Models	28
11.6	9.6 — Test-Time Compute Scaling	29
12	Phase 10 — Future DeepSeek Systems	30
12.1	10.1 — DeepSeek Agentic Architecture	30
12.2	10.2 — DeepSeek V4 Architecture	30
12.3	10.3 — Million-Token Context Windows	31
12.4	10.4 — AI Infrastructure Beyond CUDA	31
12.5	10.5 — DeepSeek vs GPT-5 vs Gemini	31
12.6	10.6 — Future of Open-Source Frontier Models	32
13	Master Resource Library	33
13.1	Official DeepSeek Papers (Complete List)	33
13.2	Foundational Papers (Read Before Writing Each Phase)	34
13.3	Essential Blogs and Online Resources	34
13.4	Code Repositories	35
14	Writing Guide & Article Template	36
14.1	The Standard Article Structure (LaTeX Template)	36
14.2	Tone, Voice, and Style Rules	36
14.3	The Publishing Checklist	37

1. How to Use This Roadmap

This document is your complete blueprint. For every article in the series you will find: (1) the exact outline following the Pyramid Model, (2) the primary papers to read, (3) the best blog explainers, (4) background foundational papers, (5) code resources, and (6) analogies to use. Read *this document* once fully before writing a single word.

1.1 The Pyramid Writing Model — Applied to Every Article

Every article in this series follows a strict 4-layer pyramid. Write in this order, always.



1.2 Difficulty Legend

Beginner = No ML background needed **Intermediate** = Undergraduate CS/Math **Advanced** = ML practitioner **Expert** = Researcher-level

1.3 Pre-Writing Checklist (Every Article)

- ☐ Read the primary paper fully (not just abstract)
- ☐ Read ≥ 2 blog explainers from the resource list
- ☐ Write your one-sentence “so what?” before anything else
- ☐ Have 3 concrete analogies ready (one from cooking/sport/city planning)
- ☐ Have ≥ 5 specific numbers/statistics ready
- ☐ Identify the #1 misconception to debunk
- ☐ Write the bridge sentence to the next article
- ☐ Open the shared $\text{\LaTeX}$ article template before typing

2. Full Series Map — All 50+ Articles

Ph.	#	Article Title	Level
Phase 1	1.1	Introduction to DeepSeek Architecture	Beginner
	1.2	How Tokens Flow Through an LLM	Beginner
	1.3	Attention Mechanism Explained	Beginner
	1.4	Self-Attention From Scratch	Intermediate
	1.5	Causal Attention & Autoregressive Generation	Intermediate
	1.6	Multi-Head Attention Internals	Intermediate
	1.7	Multi-Head Attention Implementation in Python	Intermediate
2	2.1	KV Cache Internals and Memory Bottlenecks	Intermediate
	2.2	Multi-Query Attention (MQA) Explained	Intermediate
	2.3	Grouped Query Attention (GQA) Deep Dive	Intermediate
	2.4	Why Traditional Attention Scaling Breaks	Advanced
Phase 3	3.1	Multi-Head Latent Attention (MLA) Explained	Advanced
	3.2	MLA Implementation From Scratch	Advanced
	3.3	MLA vs MQA vs GQA Comparison	Advanced
	3.4	How DeepSeek Reduced KV Cache Memory	Advanced
	3.5	DeepSeek MLA + RoPE Architecture	Advanced
	4.1	Integer Positional Encoding	Beginner
	4.2	Binary Positional Encoding	Beginner
	4.3	Sinusoidal Positional Encoding Explained	Intermediate
	4.4	Rotary Positional Encoding (RoPE) Visual Guide	Advanced
	4.5	Why RoPE Became Standard in Modern LLMs	Advanced
Phase 5	5.1	Introduction to Mixture of Experts	Intermediate
	5.2	MoE Routing and Sparse Activation	Intermediate
	5.3	Visualizing Expert Selection	Intermediate
	5.4	Auxiliary Loss and Load Balancing	Advanced
	5.5	Capacity Factor in MoE Systems	Advanced
	5.6	How DeepSeek Rewrote MoE Architecture	Advanced
	5.7	Coding MoE From Scratch in Python	Advanced
	6.1	Why Next-Token Prediction is Limited	Intermediate
	6.2	Introduction to Multi-Token Prediction	Intermediate
	6.3	How DeepSeek Improved MTP	Advanced
	6.4	MTP Training Architecture	Advanced
	6.5	MTP Implementation From Scratch	Advanced
	6.6	Speculative Decoding and Fast Generation	Advanced
Phase 7	7.1	Introduction to LLM Quantization	Intermediate
	7.2	Mixed Precision Training Explained	Intermediate
	7.3	Fine-Grained Quantization in DeepSeek	Advanced
	7.4	Accumulation Precision and Stability	Advanced
	7.5	Online Quantization Techniques	Advanced

3. Phase 1 — LLM Foundations

Phase 1 — LLM Foundations (7 Articles)

Goal: Build the reader's mental model of how a Large Language Model works — from raw text to generated tokens. No assumed ML background. These are the articles that make someone a confident reader of all future phases.

3.1 1.1 — Introduction to DeepSeek Architecture

Level: **Beginner** **Hook:** “Before you understand why DeepSeek is different, you need to know what every LLM shares.”

Outline

1. The bird's-eye view: text in → text out. What happens in between?
2. Layers of a transformer: Embedding → Attention → FFN → Output
3. DeepSeek's specific modifications (MLA, MoE) as a preview — like showing the destination before the journey
4. What makes DeepSeek architecturally unique vs GPT, LLaMA, Mistral
5. A visual map of the entire series (the 10-phase roadmap as a diagram)

Resources

- **Primary:** “Attention Is All You Need” — [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- **Blog:** The Illustrated Transformer — <https://jalammar.github.io/illustrated-transformer/>
- **Blog:** Sebastian Raschka — Understanding and Coding the Self-Attention Mechanism — <https://sebastianraschka.com/blog/2023/self-attn-pytorch-mac.html>
- **Blog:** DeepSeek-V3 Technical Report overview — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- **Code:** Andrej Karpathy's nanoGPT — <https://github.com/karpathy/nanoGPT>

Analogy to use

“An LLM is like an enormous translation office. The input text enters as one language (tokens); each floor of the building (layer) refines the understanding; the final floor outputs the next word. DeepSeek's innovation is in redesigning specific floors to be faster and cheaper.”

3.2 1.2 — How Tokens Flow Through an LLM

Level: **Beginner** **Hook:** “LLMs don't read words. They read numbers. Here's the journey from your sentence to those numbers and back.”

Outline

1. Tokenisation: BPE, WordPiece, SentencePiece — with live examples
2. Token IDs → embedding vectors (why 4096 dimensions?)
3. Forward pass: residual stream, layer norm, attention, FFN
4. Logit output → softmax → sampling strategies (greedy, top-k, temperature)
5. The autoregressive loop: how one token becomes 1000

Resources

- **Blog:** Andrej Karpathy — Let's Build the GPT Tokenizer (YouTube) <https://www.youtube.com/watch?v=zduSFxRajkE>
- **Blog:** The Illustrated GPT-2 — <https://jalammar.github.io/illustrated-gpt2/>
- **Code:** Hugging Face tokenizers library — <https://github.com/huggingface/tokenizers>

- **Interactive:** Tokenizer visualiser — <https://platform.openai.com/tokenizer>

3.3 1.3 — Attention Mechanism Explained

Level: **Beginner** **Hook:** “The attention mechanism is the one idea that changed AI forever. Here’s what it actually does.”

Outline

1. The core problem: how does a word know which other words to pay attention to?
2. The Query-Key-Value (QKV) framework in plain English
3. Dot-product similarity: why we multiply Q and K
4. Scaling factor $1/\sqrt{d_k}$: why it’s there and what breaks without it
5. The attention weight matrix as a heatmap: visualizing what the model “sees”

Resources

- **Primary:** “Attention Is All You Need” (focus on Section 3.2) — [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- **Blog:** 3Blue1Brown — Visualising Attention (YouTube) <https://www.youtube.com/watch?v=eMlx5fFNoYc>
- **Blog:** The Illustrated Transformer — Jay Alammar
- **Paper (background):** “Neural Machine Translation by Jointly Learning to Align and Translate” (Bahdanau) — [arXiv:1409.0473](https://arxiv.org/abs/1409.0473)

Analogy to use

“Attention is like writing a research paper. Your current sentence (Query) searches through your notes (Keys) to find the most relevant sources (Values) to pull forward. Heavily cited sources get weighted more.”

3.4 1.4 — Self-Attention From Scratch

Level: **Intermediate** **Hook:** “Stop using attention as a black box. Here’s every matrix multiplication, explained.”

Outline

1. Input: token embedding matrix $X \in \mathbb{R}^{n \times d}$
2. Projection matrices: $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$
3. Computing $Q = XW_Q, K = XW_K, V = XW_V$
4. Attention scores: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
5. Walk through a 4×4 numeric example by hand
6. What the weight matrix means intuitively
7. Python implementation from scratch (NumPy, then PyTorch)

Resources

- **Code:** Andrej Karpathy — “Let’s Build GPT from Scratch” (YouTube) <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- **Blog:** Sebastian Raschka — Self-Attention from scratch in PyTorch
- **Code:** Raschka’s GitHub — <https://github.com/rasbt/LLMs-from-scratch>
- **Book:** “Build a Large Language Model From Scratch” — Sebastian Raschka (Manning)

3.5 1.5 — Causal Attention and Autoregressive Generation

Level: **Intermediate** **Hook:** “Why can’t an LLM see into the future? The causal mask is why — and it’s elegantly simple.”

Outline

1. The problem: in training, we have the full sequence; in inference, we only have the past
2. Causal (triangular) attention mask: upper triangle set to $-\infty$
3. Effect on softmax: future tokens get probability 0
4. Autoregressive generation: the inference loop step by step
5. Teacher forcing during training vs free-running inference
6. Why this design forces the KV cache to be recomputed — motivating Phase 2

Resources

- **Code:** Karpathy nanoGPT — <https://github.com/karpathy/nanoGPT> (the mask in `model.py`)
- **Book:** Raschka — “Build a Large Language Model From Scratch” (Chapter 3)
- **Blog:** The Illustrated GPT-2 (autoregressive section) — Jay Alammar

3.6 1.6 — Multi-Head Attention Internals

Level: **Intermediate** **Hook:** “One attention head learns one pattern. Why does GPT-4 have 96 of them?”

Outline

1. Motivation: single attention learns one type of relationship (syntactic, semantic, positional)
2. Multi-head: split d into h heads of size d/h ; run attention in parallel
3. Concatenation and output projection W_O
4. What different heads learn (visualisations from BertViz)
5. Computational cost: $O(n^2d)$ — the quadratic problem
6. Head dimensions: GPT-3 (96 heads, $d = 12288$); DeepSeek-V2 (128 heads, $d = 5120$)

Resources

- **Primary:** “Attention Is All You Need” (Section 3.2.2) — [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- **Tool:** BertViz — attention visualiser — <https://github.com/jessevig/bertviz>
- **Blog:** Lena Voita — Attention Heads analysis — https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

3.7 1.7 — Multi-Head Attention Implementation in Python

Level: **Intermediate** **Hook:** “Theory is nice. Running code is better. Let’s implement MHA in 50 lines of PyTorch.”

Outline

1. Full PyTorch MHA class from scratch (no `nn.MultiheadAttention`)
2. Efficient batched QKV projection with a single weight matrix
3. Reshaping for multi-head: $(B, T, C) \rightarrow (B, nh, T, hs)$
4. Applying the causal mask
5. Dropout on attention weights
6. Testing: verify output shapes, verify causality
7. Performance profiling: FLOPs, memory usage

Resources

- **Code:** Karpathy nanoGPT `model.py` — <https://github.com/karpathy/nanoGPT/blob/master/model.py>
- **Code:** Raschka LLMs-from-scratch — <https://github.com/rasbt/LLMs-from-scratch>
- **Docs:** PyTorch `scaled_dot_product_attention` — <https://pytorch.org/docs/stable/generated/>

[torch.nn.functional.scaled_dot_product_attention.html](#)

4. Phase 2 — KV Cache & Efficient Attention

Phase 2 — KV Cache & Efficient Attention (4 Articles)

Goal: Explain the #1 memory bottleneck in LLM inference (the KV cache) and trace the evolution of attention variants that try to solve it. This phase sets up why MLA (Phase 3) is necessary and brilliant.

4.1 2.1 — KV Cache Internals and Memory Bottlenecks

Level: **Intermediate** **Hook:** “Running a 70B LLM costs 140GB just for weights. The KV cache doubles that. Here’s why.”

Outline

1. What is the KV cache? Why we cache instead of recomputing
2. Memory formula: $2 \times n_layers \times seq_len \times n_heads \times head_dim \times bytes$
3. Worked example: LLaMA-2 70B at 4096 context = how many GB?
4. How KV cache grows with batch size and context length
5. The fundamental tension: longer context = smarter model = more memory
6. Why this led to MQA, GQA, and ultimately MLA

Resources

- **Blog:** “Transformers KV Caching Explained” — <https://neptune.ai/blog/llm-kv-cache>
- **Blog:** Hugging Face blog on KV cache — <https://huggingface.co/blog/kv-cache-quantization>
- **Paper:** PagedAttention (vLLM) — [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- **Tool:** LLM Memory Calculator — <https://huggingface.co/spaces/hysts/llm-memory-estimator>

4.2 2.2 — Multi-Query Attention (MQA) Explained

Level: **Intermediate** **Hook:** “What if all attention heads shared the same keys and values? MQA tried exactly that.”

Outline

1. MHA recap: h heads, each with own $K, V \rightarrow h \times$ memory
2. MQA (Shazeer 2019): 1 shared K and V across all heads; h separate Q
3. Memory saving: KV cache shrinks by factor h
4. Quality cost: benchmarks show degradation on complex tasks
5. Where MQA is used: PaLM, Falcon, early Mistral
6. The quality-efficiency trade-off that motivated GQA

Resources

- **Paper:** “Fast Transformer Decoding: One Write-Head is All You Need” (Shazeer) — [arXiv:1911.02150](https://arxiv.org/abs/1911.02150)
- **Blog:** Lilian Weng — Large Transformer Model Inference Optimization — <https://lilianweng.github.io/posts/2023-01-10-inference-optimization/>

4.3 2.3 — Grouped Query Attention (GQA) Deep Dive

Level: **Intermediate** **Hook:** “GQA is the compromise between MHA’s quality and MQA’s speed. Here’s the exact trade-off.”

Outline

1. GQA: group h heads into g groups; each group shares K, V

2. MHA = GQA with $g = h$; MQA = GQA with $g = 1$
3. Memory formula: KV cache reduces by factor h/g
4. Quality vs speed: GQA nearly matches MHA quality at MQA speed
5. Where GQA is used: LLaMA-2/3, Mistral, Gemma, Qwen
6. Why GQA still isn't the final answer (motivates MLA)

Resources

- **Paper:** “GQA: Training Generalised Multi-Query Transformer” — [arXiv:2305.13245](#)
- **Blog:** GQA visual explanation — <https://dipkumar.dev/becoming-the-unbeatable/posts/gpt-kvcache/>

4.4 2.4 — Why Traditional Attention Scaling Breaks

Level: **Advanced** **Hook:** “MHA’s memory cost scales with $O(n \cdot d \cdot L)$. At 1M tokens, that’s catastrophic.”

Outline

1. The $O(n^2)$ compute problem (FlashAttention solves this — cite it)
2. The $O(n \cdot h \cdot d_h \cdot L)$ memory problem (no one had solved this — until MLA)
3. Quantitative comparison: MHA vs MQA vs GQA vs MLA at 128K context
4. Why KV cache quantisation is a band-aid, not a cure
5. The insight DeepSeek had: compress into a latent vector, not reduce heads

Resources

- **Paper:** FlashAttention-2 — [arXiv:2307.08691](#)
- **Paper:** FlashAttention-3 — [arXiv:2407.08608](#)
- **Paper:** DeepSeek-V2 (Section 2 on MLA motivation) — [arXiv:2405.04434](#)

5. Phase 3 — DeepSeek MLA (Major Innovation)

Phase 3 — Multi-Head Latent Attention (5 Articles)

Goal: This is the centrepiece technical innovation of DeepSeek-V2/V3. Give it the depth it deserves. By the end of Phase 3, the reader understands MLA better than most ML engineers.

5.1 3.1 — Multi-Head Latent Attention (MLA) Explained

Level: **Advanced** **Hook:** “DeepSeek compressed the KV cache by 93% without losing quality. The trick: low-rank projection.”

Outline

1. The core idea: instead of caching full K and V per head, cache a single low-rank latent vector c^{KV}
2. MLA projection: $c^{KV} = W^{DKV} h_t$ where $W^{DKV} \in \mathbb{R}^{d_c \times d}$, $d_c \ll d \cdot n_h$
3. At inference: project c^{KV} back up to get K and V on demand
4. Why this works: low-rank structure in attention matrices is empirically validated
5. Memory saving: from $n_h \cdot d_h$ per token to d_c per token
6. DeepSeek-V2 numbers: $d_c = 512$ vs full 5120 — $10\times$ reduction

Resources

- **Primary:** DeepSeek-V2 Technical Report (Section 2) — [arXiv:2405.04434](https://arxiv.org/abs/2405.04434)
- **Blog:** “TransMLA: Multi-Head Latent Attention Is All You Need” — [arXiv:2502.07864](https://arxiv.org/abs/2502.07864)
- **Blog:** MLA explained — <https://kippl.ly/transformer-inference-arithmetic/>
- **Paper:** Towards Economical Inference: Enabling MLA in Any LLM — [arXiv:2502.14837](https://arxiv.org/abs/2502.14837)

Analogy to use

“MHA is like recording every meeting in full 4K video (the KV cache). MLA is like keeping only a compressed meeting summary (the latent vector), then expanding it back to full fidelity when needed. You lose almost nothing, but the filing cabinet is $10\times$ smaller.”

5.2 3.2 — MLA Implementation From Scratch

Level: **Advanced** **Hook:** “The paper describes it. Here we build it. Every line explained.”

Outline

1. Full PyTorch implementation of MLA
2. W_{DKV} : the downprojection (compress to latent)
3. W_{UK} , W_{UV} : the upprojections (expand K and V from latent)
4. The query latent vector c^Q and its upprojection
5. Handling RoPE decoupling (see Article 3.5 for details)
6. Inference mode vs training mode: the caching strategy
7. Memory profiling: before and after MLA
8. Unit tests: verify output matches reference MHA on synthetic data

Resources

- **Code:** DeepSeek-V2 official code — <https://github.com/deepseek-ai/DeepSeek-V2>
- **Code:** HuggingFace DeepSeek-V3 modelling — https://github.com/huggingface/transformers/blob/main/src/transformers/models/deepseek_v3/

- **Paper:** MHA2MLA transition method — [arXiv:2502.14837](#)

5.3 3.3 — MLA vs MQA vs GQA Comparison

Level: **Advanced** **Hook:** “Three solutions to the same problem. One is clearly better. Here’s the data.”

Outline

1. Side-by-side architecture diagrams: MHA, MQA, GQA, MLA
2. KV cache memory comparison (formula + numbers for each)
3. Quality benchmarks: MMLU, MATH, code — where does each variant degrade?
4. Inference speed comparison: prefill vs decoding throughput
5. When each is the right choice
6. Why major labs (Meta, Google, Mistral) have not yet adopted MLA

Resources

- **Paper:** MHA paper — [arXiv:1706.03762](#)
- **Paper:** MQA paper — [arXiv:1911.02150](#)
- **Paper:** GQA paper — [arXiv:2305.13245](#)
- **Paper:** MLA paper — [arXiv:2405.04434](#)
- **Paper:** TransMLA comparison — [arXiv:2502.07864](#)

5.4 3.4 — How DeepSeek Reduced KV Cache Memory

Level: **Advanced** **Hook:** “Running DeepSeek-V3 at 128K context: how much memory does the KV cache actually use?”

Outline

1. DeepSeek-V2 KV cache: theoretical formula vs MHA baseline
2. Worked numerical example: batch=1, context=128K, 28 layers
3. Comparison table: MHA (LLaMA-3) vs GQA (LLaMA-3) vs MLA (DeepSeek-V3)
4. Effect on serving cost: throughput, latency, GPU hours
5. How MLA enables longer context at the same hardware budget
6. Impact on V4’s 1M-token context feasibility

Resources

- **Paper:** DeepSeek-V3 Technical Report (Section 2) — [arXiv:2412.19437](#)
- **Paper:** Insights into DeepSeek-V3: Scaling Challenges — [arXiv:2505.09343](#)
- **Tool:** LLM Memory Calculator — <https://huggingface.co/spaces/hysts/llm-memory-estimator>

5.5 3.5 — DeepSeek MLA + RoPE Architecture

Level: **Advanced** **Hook:** “RoPE and MLA don’t naturally work together. DeepSeek had to decouple them. Here’s how.”

Outline

1. The conflict: RoPE is applied to the full K , but MLA compresses K into a latent
2. Decoupled RoPE: split K into content part (from latent) + positional part (RoPE-only)
3. Extra RoPE query/key heads: q_t^R and k_t^R carry position without entering the latent
4. How this is implemented in DeepSeek-V2/V3 code
5. Why this is necessary (what breaks if you don’t decouple)
6. The additional memory cost of RoPE keys vs savings from MLA

Resources

- **Paper:** DeepSeek-V2 (Section 2.1, Decoupled RoPE) — [arXiv:2405.04434](#)
- **Paper:** RoPE original paper — [arXiv:2104.09864](#)
- **Code:** DeepSeek-V2 MLA with RoPE — <https://github.com/deepseek-ai/DeepSeek-V2/blob/main/inference/model.py>

6. Phase 4 — Positional Encoding Evolution

Phase 4 — Positional Encoding Evolution (5 Articles)

Goal: Transformers have no inherent sense of word order. This phase traces the full history of positional encodings from the simplest integer scheme to RoPE — the modern standard used by DeepSeek, LLaMA, Mistral, and Qwen.

6.1 4.1 — Integer Positional Encoding

Level: **Beginner** **Hook:** “The most obvious solution: just number the positions. Obvious — and wrong. Here’s why.”

Outline

1. The problem: transformers process all tokens in parallel — they lose position information
2. Naive idea: append position number (1, 2, 3...) to each embedding
3. Why this fails: position 1000 looks completely different from position 1 numerically
4. No generalisation to sequences longer than seen in training
5. This motivates the search for a better representation

6.2 4.2 — Binary Positional Encoding

Level: **Beginner** **Hook:** “Binary is how computers count. Could it work for position encoding? Almost.”

Outline

1. Represent position as binary string (position 5 = 0101)
2. Better: each bit varies at a different frequency
3. Still discontinuous: position 7 (0111) and position 8 (1000) are maximally different
4. Motivates a smooth, continuous version — sinusoidal encoding

6.3 4.3 — Sinusoidal Positional Encoding Explained

Level: **Intermediate** **Hook:** “The original Transformer paper used sine and cosine waves for position. Elegant — but limited.”

Outline

1. Definition: $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$
2. Visual: the encoding as a striped image — low frequencies for long-range, high for short
3. Properties: unique per position, smooth, relative positions encodable via dot product
4. Limitation: fixed at training; does not generalise to longer sequences
5. Why learned absolute PE (BERT, GPT-2) also has the same limitation

Resources

- **Primary:** “Attention Is All You Need” (Section 3.5) — [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- **Blog:** The Annotated Transformer — <https://nlp.seas.harvard.edu/annotated-transformer/>
- **Blog:** Amirhossein Kazemnejad — Positional Encoding from Scratch https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

6.4 4.4 — Rotary Positional Encoding (RoPE) Visual Guide

Level: **Advanced** **Hook:** “RoPE encodes position by rotating vectors in complex space. It’s beautiful and it works.”

Outline

1. Core insight: encode position as a rotation applied to Q and K vectors
2. 2D case: rotating a vector by angle $m\theta$ for position m
3. Inner product between rotated Q and K depends only on their *relative* position — key property
4. Extension to d -dimensional space: pair up dimensions, each pair gets a different θ_i
5. Mathematical formulation: $f(\mathbf{x}, m) = \mathbf{x}e^{im\theta}$
6. RoPE vs absolute PE: generalisation to unseen lengths, relative position awareness

Resources

- **Primary:** RoPE Paper — “RoFormer: Enhanced Transformer with Rotary Position Embedding” — [arXiv:2104.09864](https://arxiv.org/abs/2104.09864)
- **Blog:** Eleuther AI — RoPE explained — <https://blog.eleuther.ai/rotary-embeddings/>
- **Blog:** “Rotary Positional Embeddings” — <https://fleetwood.dev/posts/understanding-RoPE>
- **Code:** RoPE in LLaMA — <https://github.com/meta-llama/llama/blob/main/llama/model.py>

6.5 4.5 — Why RoPE Became Standard in Modern LLMs

Level: **Advanced** **Hook:** “LLaMA, Mistral, Qwen, Gemma, and DeepSeek all use RoPE. This is why.”

Outline

1. Comparison table: absolute PE vs relative PE (T5) vs RoPE vs ALiBi
2. RoPE’s winning properties: relative position, extrapolation, no added parameters
3. Context length extension tricks built on RoPE: YaRN, LongRoPE, NTK-aware scaling
4. How DeepSeek extends to 128K context using YaRN on top of RoPE
5. How V4 reaches 1M context with hybrid attention + modified RoPE

Resources

- **Paper:** YaRN (long-context RoPE extension) — [arXiv:2309.00071](https://arxiv.org/abs/2309.00071)
- **Paper:** LongRoPE — [arXiv:2402.13753](https://arxiv.org/abs/2402.13753)
- **Paper:** ALiBi (alternative to RoPE) — [arXiv:2108.12409](https://arxiv.org/abs/2108.12409)
- **Paper:** DeepSeek-V3 long context section — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)

7. Phase 5 — Mixture of Experts (MoE)

Phase 5 — Mixture of Experts (7 Articles)

Goal: MoE is the second major architectural pillar of DeepSeek. This phase builds from zero (what is an expert?) to a full implementation and analysis of DeepSeekMoE’s innovations: fine-grained segmentation, shared experts, and auxiliary-loss-free load balancing.

7.1 5.1 — Introduction to Mixture of Experts

Level: **Intermediate** **Hook:** “Instead of one giant brain, use 64 specialist brains and consult only the relevant ones.”

Outline

1. The FFN layer in a standard transformer (dense — all neurons fire)
2. MoE idea: replace FFN with N expert FFNs; only activate k
3. Why this works: parameter count $\uparrow$, compute $\approx$ constant
4. History: Jordan & Jacobs (1991) $\rightarrow$ Shazeer et al. (2017) $\rightarrow$ GShard $\rightarrow$ Switch $\rightarrow$ DeepSeek
5. The scaling law argument: more parameters = smarter, but we need efficiency

Resources

- **Paper:** Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer (Shazeer) — [arXiv:1701.06538](https://arxiv.org/abs/1701.06538)
- **Paper:** Switch Transformer — [arXiv:2101.03961](https://arxiv.org/abs/2101.03961)
- **Blog:** Hugging Face — Mixture of Experts Explained — <https://huggingface.co/blog/moe>

Analogy to use

“A hospital has 100 specialists. When you arrive, a triage nurse (the router) sends you to the 2 most relevant doctors. You get expert care without all 100 doctors examining you.”

7.2 5.2 — MoE Routing and Sparse Activation

Level: **Intermediate** **Hook:** “The hardest problem in MoE: how does the model decide which experts to use?”

Outline

1. The router: a learned linear layer that produces logits over N experts
2. Top- k selection: choose the k experts with highest router scores
3. Gating: weight expert outputs by their router scores (softmax over top- k)
4. Token-choice vs expert-choice routing
5. The collapse problem: why all tokens gravitate to the same 2-3 experts
6. Load balancing as the fundamental challenge (motivates Articles 5.4 and 5.5)

Resources

- **Paper:** Mixture of Experts with Expert Choice (Zhou et al.) — [arXiv:2202.09368](https://arxiv.org/abs/2202.09368)
- **Blog:** “Everything you need to know about MoE” — <https://huggingface.co/blog/moe>

7.3 5.3 — Visualizing Expert Selection

Level: **Intermediate** **Hook:** “Which experts does DeepSeek actually use for maths vs code vs poetry? The answer is fascinating.”

Outline

1. Building a heatmap: for each domain, which experts are activated?
2. Evidence of specialisation: some experts strongly prefer certain domains
3. Expert utilisation histograms: ideally flat (all experts used equally)
4. What “expert collapse” looks like in a utilisation histogram
5. Code: logging expert selection during inference with a HuggingFace model

Resources

- **Paper:** “Examining Post-Training Quantization for Mixture-of-Experts” — expert analysis section
- **Code:** Mixtral MoE routing analysis — <https://github.com/mistralai/mistral-src>
- **Blog:** Mixtral of Experts paper — [arXiv:2401.04088](https://arxiv.org/abs/2401.04088)

7.4 5.4 — Auxiliary Loss and Load Balancing

Level: **Advanced** **Hook:** “If you don’t force load balancing, MoE models collapse. The fix — until DeepSeek changed everything.”

Outline

1. Why load imbalance happens: gradient descent prefers the already-good experts
2. Switch Transformer auxiliary loss: penalise when expert load distribution is uneven
3. Formula: $L_{aux} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$
4. The problem with auxiliary loss: it degrades model quality (conflicting objectives)
5. Hyperparameter sensitivity: α too high = overriding the language objective
6. This sets up DeepSeek’s auxiliary-loss-free approach (Articles 5.6, 8.3)

Resources

- **Paper:** Switch Transformer (load balancing section) — [arXiv:2101.03961](https://arxiv.org/abs/2101.03961)
- **Paper:** GShard — [arXiv:2006.16668](https://arxiv.org/abs/2006.16668)
- **Paper:** DeepSeekMoE — [arXiv:2401.06066](https://arxiv.org/abs/2401.06066)

7.5 5.5 — Capacity Factor in MoE Systems

Level: **Advanced** **Hook:** “The capacity factor is the hidden knob that determines whether your MoE model actually trains stably.”

Outline

1. Capacity factor definition: max tokens per expert = $C \cdot (T/N)$
2. What happens when capacity is exceeded: token dropping
3. Low C : efficient but loses information; high C : quality but memory waste
4. Expert-choice routing eliminates token dropping — at a cost
5. How DeepSeek-V3 avoids token dropping via auxiliary-loss-free balancing

Resources

- **Paper:** Switch Transformer (capacity factor analysis) — [arXiv:2101.03961](https://arxiv.org/abs/2101.03961)
- **Paper:** Mixtral of Experts — [arXiv:2401.04088](https://arxiv.org/abs/2401.04088)

7.6 5.6 — How DeepSeek Rewrote MoE Architecture

Level: **Advanced** **Hook:** “DeepSeekMoE has two ideas that no one else had. They make all the difference.”

Outline

1. **Innovation 1 — Fine-grained expert segmentation:** Instead of N large experts, use mN smaller experts. More routing choices, richer combinations.
2. **Innovation 2 — Shared experts:** K_s experts always activated for every token (handle common knowledge); K_r routed experts handle specialised tasks
3. How this differs from Switch/Mixtral: granularity + shared foundation
4. DeepSeek-V3 numbers: 256 routed experts, 1 always-active shared expert, top-8 routing
5. Auxiliary-loss-free load balancing (V3): bias terms on router logits, adjusted during training
6. Evidence that DeepSeekMoE achieves better quality at same cost vs standard MoE

Resources

- **Primary:** DeepSeekMoE paper — [arXiv:2401.06066](https://arxiv.org/abs/2401.06066)
- **Primary:** DeepSeek-V3 Technical Report (Section 2.2) — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- **Blog:** Sebastian Raschka on DeepSeekMoE — <https://magazine.sebastianraschka.com/p/technical-deepseek>

7.7 5.7 — Coding MoE From Scratch in Python

Level: **Advanced** **Hook:** “The best way to understand DeepSeekMoE is to build it. Let’s code it.”

Outline

1. Expert class: a standard FFN (SwiGLU activation as used in DeepSeek)
2. Router class: linear layer + top- k selection
3. MoE layer: dispatch tokens to experts, aggregate outputs
4. Shared expert: always-on FFN concatenated with routed output
5. Training test: does loss decrease? Do experts specialise?
6. Memory and FLOPs analysis: verify the efficiency claim

Resources

- **Code:** DeepSeek-V3 official implementation — <https://github.com/deepseek-ai/DeepSeek-V3>
- **Code:** Mixtral reference MoE — <https://github.com/mistralai/mistral-src>
- **Blog:** “Build a MoE from Scratch” — <https://huggingface.co/blog/moe>

8. Phase 6 — Multi-Token Prediction (MTP)

Phase 6 — Multi-Token Prediction (6 Articles)

Goal: MTP is DeepSeek-V3’s third major innovation. It predicts multiple future tokens simultaneously, improving training efficiency and enabling speculative decoding at inference. This phase explains it from first principles to implementation.

8.1 6.1 — Why Next-Token Prediction is Limited

Level: **Intermediate** **Hook:** “Every LLM is trained to predict just the next word. But what if it predicted the next 4?”

Outline

1. Standard LLM objective: $\mathcal{L} = -\sum_t \log P(x_t | x_{<t})$
2. The problem: predicting token $t+1$ doesn’t directly help plan tokens $t+2, t+3$
3. Evidence: LLMs are bad at “thinking ahead” (poem structure, code function signatures)
4. What would it mean to predict k tokens ahead?
5. Research background: Medusa, Speculative Decoding, early MTP attempts

Resources

- **Paper:** “Better & Faster Large Language Models via Multi-token Prediction” (Meta) — [arXiv:2404.19737](#)
- **Paper:** Medusa — [arXiv:2401.10774](#)

8.2 6.2 — Introduction to Multi-Token Prediction

Level: **Intermediate** **Hook:** “Meta proved MTP improves code generation. DeepSeek made it a first-class training objective.”

Outline

1. MTP training: predict $x_{t+1}, x_{t+2}, \dots, x_{t+k}$ from position t
2. Independent head formulation: k separate prediction heads on top of the model
3. Loss: $\mathcal{L}_{MTP} = \frac{1}{k} \sum_{i=1}^k \mathcal{L}_i$
4. Why MTP improves representations: forces the model to plan ahead
5. Training cost increase vs quality gain trade-off

Resources

- **Paper:** Meta MTP paper — [arXiv:2404.19737](#)
- **Paper:** DeepSeek-V3 Technical Report (Section 2.3) — [arXiv:2412.19437](#)

8.3 6.3 — How DeepSeek Improved MTP

Level: **Advanced** **Hook:** “DeepSeek-V3 didn’t just use MTP. It redesigned it with a sequential module architecture.”

Outline

1. DeepSeek’s MTP: D sequential MTP modules, each predicts one additional future token
2. Module architecture: embedding + transformer block + output head, sharing input embedding and output head with the main model
3. Key difference from independent heads: each module takes as input the previous module’s representation
4. Why sequential is better than independent: richer signal propagation

5. Training: MTP loss weighted by λ_{MTP} added to main loss
6. At inference: the MTP heads serve as draft tokens for speculative decoding

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 2.3 and 3.3) — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)

8.4 6.4 — MTP Training Architecture

Level: **Advanced** **Hook:** “The training graph for MTP is deceptively tricky. Here’s how to do it without gradient explosions.”

Outline

1. Tensor shapes at each MTP module
2. Gradient flow: does the MTP loss backpropagate into the main transformer?
3. Shared vs separate parameters for embedding and head
4. Loss weighting schedule: constant λ vs curriculum
5. Gradient clipping and stability tricks
6. Ablation from DeepSeek-V3: MTP off vs on (perplexity, code, math)

8.5 6.5 — MTP Implementation From Scratch

Level: **Advanced** **Hook:** “Code it: add MTP modules to a nanoGPT-style model in under 100 lines.”

Outline

1. Extend nanoGPT with 2 MTP modules
2. Custom loss computation function
3. Training loop modification: single forward pass, multiple losses
4. Verification: does the model generate better code/structure with MTP?

Resources

- **Code:** DeepSeek-V3 MTP implementation — <https://github.com/deepseek-ai/DeepSeek-V3/blob/main/inference/model.py>
- **Base:** Karpathy nanoGPT — <https://github.com/karpathy/nanoGPT>

8.6 6.6 — Speculative Decoding and Fast Generation

Level: **Advanced** **Hook:** “MTP gives you a draft token for free. Speculative decoding makes it 2–3× faster.”

Outline

1. Speculative decoding basics: draft model proposes k tokens; verifier accepts/rejects in one pass
2. Without MTP: need a separate smaller draft model
3. With MTP: the MTP heads *are* the draft model — no extra parameters
4. Acceptance rate analysis: when do MTP drafts get accepted?
5. DeepSeek-V3 inference speedup: 1.8× with 1 MTP module
6. Batch size interaction: speculative decoding is most effective at batch size 1

Resources

- **Paper:** Speculative Decoding — [arXiv:2211.17192](https://arxiv.org/abs/2211.17192)
- **Paper:** SpecInfer — [arXiv:2305.09781](https://arxiv.org/abs/2305.09781)
- **Blog:** Hugging Face — Speculative Decoding — <https://huggingface.co/blog/speculative-decoding>

9. Phase 7 — Quantization & Inference Optimization

Phase 7 — Quantization & Inference Optimization (6 Articles)

Goal: DeepSeek-V3 was the first model at this scale to use FP8 training. This phase explains quantisation from first principles through to the fine-grained tile-based FP8 system DeepSeek engineered.

9.1 7.1 — Introduction to LLM Quantization

Level: **Intermediate** **Hook:** “A 70B model in FP32 needs 280GB. In 4-bit, it fits on a gaming PC. Here’s how.”

Outline

1. Data types: FP32, FP16, BF16, INT8, INT4, FP8
2. Quantisation basics: map a range of floats to a smaller set of values
3. Post-training quantisation (PTQ) vs quantisation-aware training (QAT)
4. GPTQ, AWQ, LLM.int8() — the landscape of inference quantisation
5. Accuracy-efficiency trade-off: when does quantisation hurt?
6. Why training quantisation (FP8) is much harder than inference quantisation

Resources

- **Paper:** GPTQ — [arXiv:2210.17323](https://arxiv.org/abs/2210.17323)
- **Paper:** AWQ: Activation-aware Weight Quantization — [arXiv:2306.00978](https://arxiv.org/abs/2306.00978)
- **Blog:** HuggingFace Quantisation Guide — <https://huggingface.co/docs/transformers/quantization>
- **Blog:** LLM.int8() paper — [arXiv:2208.07339](https://arxiv.org/abs/2208.07339)

9.2 7.2 — Mixed Precision Training Explained

Level: **Intermediate** **Hook:** “NVIDIA’s A100 is 3× faster in BF16 than FP32. Mixed precision exploits this without losing accuracy.”

Outline

1. FP32 vs BF16: exponent bits vs mantissa bits, why BF16 is preferred over FP16 for training
2. Mixed precision training: forward/backward in BF16, master weights in FP32
3. Loss scaling: preventing underflow in FP16 gradients
4. The NVIDIA paper that introduced mixed precision training
5. What goes wrong without careful implementation: gradient explosion, NaN
6. How BF16 became the baseline — and why DeepSeek pushed to FP8

Resources

- **Paper:** Mixed Precision Training (Micikevicius et al.) — [arXiv:1710.03740](https://arxiv.org/abs/1710.03740)
- **Docs:** PyTorch AMP (Automatic Mixed Precision) — <https://pytorch.org/docs/stable/amp.html>

9.3 7.3 — Fine-Grained Quantization in DeepSeek

Level: **Advanced** **Hook:** “The problem with FP8: outlier activations destroy the quantisation range. DeepSeek’s solution: quantise in tiny tiles.”

Outline

1. The outlier problem: a single large activation value ruins the quantisation scale for the whole tensor
2. Coarse-grained quantisation: one scale factor per tensor — fast but inaccurate
3. Fine-grained: one scale factor per 1×128 tile of activations
4. DeepSeek-V3 specifics: weight quantisation per 128×128 blocks
5. How tile-level scaling captures local outliers without global damage
6. Why this requires custom CUDA kernels

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3.3) — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- **Paper:** FP8 Formats for Deep Learning (Micikevicius et al.) — [arXiv:2209.05433](https://arxiv.org/abs/2209.05433)

9.4 7.4 — Accumulation Precision and Stability

Level: **Advanced** **Hook:** “FP8 matrix multiplies accumulate in FP32. Skip this and your model silently loses accuracy. DeepSeek almost learned this the hard way.”

Outline

1. Why matmul accumulation precision matters: small errors compound through d -dimensional sums
2. NVIDIA H800 FP8 GEMM: accumulates in FP32 only every 128 elements by default
3. DeepSeek’s discovery: this default causes precision loss for large d
4. Fix: force promotion to FP32 accumulation after every 128-element partial sum
5. The custom CUDA kernel that implements this
6. Quantitative impact on training stability and final model quality

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3.3.2) — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- **Docs:** NVIDIA H100 FP8 training guide — <https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/>

9.5 7.5 — Online Quantization Techniques

Level: **Advanced** **Hook:** “Offline quantisation computes scales once. Online quantisation computes them every forward pass. DeepSeek uses online.”

Outline

1. Offline (static) quantisation: calibration dataset, fixed scales
2. Online (dynamic) quantisation: compute max per tile at runtime
3. The trade-off: online is slower but handles distribution shift during training
4. Why training requires online: activation distributions change every step
5. Inference can use offline: distributions are fixed after training
6. DeepSeek-V3’s hybrid: online for activations, offline for weights

9.6 7.6 — FP8 Training and Inference Scaling

Level: **Expert** **Hook:** “DeepSeek-V3 was the first model to successfully train 671B parameters with FP8. This is how they did it.”

Outline

1. Full FP8 training pipeline: which operations in FP8, which stay in BF16/FP32
2. Operators that *cannot* be FP8: embedding, layer norm, attention softmax, loss

3. Communication: gradients sent between GPUs in BF16 (not FP8) for stability
4. Scaling laws for FP8: does the quality match BF16 training?
5. Inference: FP8 weights + FP8 activations = $4\times$ throughput vs FP32
6. Hardware requirements: FP8 only on H100, H800 (not A100)
7. Cost calculation: DeepSeek-V3 training cost breakdown

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3) — [arXiv:2412.19437](#)
- **Paper:** “Insights into DeepSeek-V3: Scaling Challenges” — [arXiv:2505.09343](#)
- **Paper:** “FP8 versus BF16 for LLM Pre-Training” — [arXiv:2309.17224](#)
- **Blog:** NVIDIA Transformer Engine — <https://github.com/NVIDIA/TransformerEngine>

10. Phase 8 — DeepSeek V2/V3 System Design

Phase 8 — DeepSeek V2/V3 System Design (8 Articles)

Goal: How do you actually train a 671B MoE model on 2048 H800 GPUs? This phase covers the systems engineering: parallelism strategies, communication topology, memory optimisation, and the infrastructure choices that made the \$5.58M training budget possible.

10.1 8.1 — DeepSeek V2 Architecture Breakdown

Level: **Advanced** **Hook:** “236B parameters, 21B active. Here’s every design decision in DeepSeek-V2.”

Outline

1. Full architecture table: layers, hidden dim, attention heads, experts
2. MLA configuration: $d_c = 512$, $d^R = 64$ for RoPE decoupled keys
3. DeepSeekMoE configuration: 160 experts, top-6, 2 shared experts
4. Training data: 8.1T tokens, mixture breakdown
5. SFT and RLHF post-training pipeline
6. Benchmark results: where V2 beats LLaMA-3 70B

Resources

- **Primary:** DeepSeek-V2 Technical Report — [arXiv:2405.04434](https://arxiv.org/abs/2405.04434)

10.2 8.2 — DeepSeek V3 Architecture Explained

Level: **Advanced** **Hook:** “671B total, 37B active, 14.8T tokens, \$5.58M. Every number explained.”

Outline

1. Full architecture table vs V2: what changed
2. MoE: 256 routed experts + 1 shared, top-8 routing
3. New V3-only features: auxiliary-loss-free balancing, MTP (2 modules)
4. FP8 training (first at this scale)
5. DualPipe parallelism strategy
6. Multi-Token Prediction with $D = 2$ modules
7. Post-training: SFT on 1.5M examples, GRPO RL

Resources

- **Primary:** DeepSeek-V3 Technical Report — [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- **Blog:** Sebastian Raschka DeepSeek Technical Tour — <https://magazine.sebastianraschka.com/p/technical-deepseek>

10.3 8.3 — Auxiliary-Loss-Free Load Balancing

Level: **Advanced** **Hook:** “Every other MoE model fights load imbalance with an auxiliary loss that hurts quality. DeepSeek eliminated it.”

Outline

1. Recap: why auxiliary loss is harmful (conflicting gradients)
2. DeepSeek-V3 solution: learnable bias terms b_i added to each expert’s router logit
3. Bias update rule: if expert i is overloaded, decrease b_i ; if underloaded, increase

4. This happens outside the gradient graph — no conflict with the language loss
5. Sequence-level balance: V3 additionally enforces balance across the full sequence
6. Empirical result: equal or better expert utilisation vs auxiliary loss, with better model quality

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 2.2.3) — [arXiv:2412.19437](#)

10.4 8.4 — Sparse Compute Economics

Level: **Intermediate** **Hook:** “DeepSeek-V3 is 671B parameters but costs the same to run as a 37B dense model. The economics of sparsity.”

Outline

1. FLOP math: dense 37B model vs sparse 671B/37B active
2. Why active parameters $\approx$ compute cost (FLOPs)
3. Why total parameters $\approx$ memory cost
4. The MoE scaling law: can you keep growing parameters without growing FLOPs?
5. Diminishing returns: does sparsity always win?
6. Cost comparison table: DeepSeek-V3 vs LLaMA-3 70B vs GPT-4 (estimated)

Resources

- **Paper:** Scaling Laws for Neural Language Models — [arXiv:2001.08361](#)
- **Paper:** Unified Scaling Laws for Routed Language Models — [arXiv:2202.01855](#)

10.5 8.5 — Distributed Training for MoE Models

Level: **Expert** **Hook:** “Training DeepSeek-V3 used 4 types of parallelism simultaneously. Here’s how they fit together.”

Outline

1. The 4 parallelisms: data, tensor, pipeline, expert
2. Data parallelism: replicate model, split data
3. Tensor parallelism: split weight matrices across GPUs (expensive communication)
4. Pipeline parallelism: split model layers across GPUs (bubble problem)
5. Expert parallelism: route tokens across GPUs to their assigned experts
6. DeepSeek-V3’s configuration: TP=1, PP=8, EP=64 for MoE layers
7. DualPipe: DeepSeek’s custom pipeline schedule that overlaps compute and communication

Resources

- **Paper:** Megatron-LM — [arXiv:1909.08053](#)
- **Paper:** GPipe — [arXiv:1811.06965](#)
- **Paper:** DeepSeek-V3 Technical Report (Section 3.2) — [arXiv:2412.19437](#)
- **Blog:** Lilian Weng — Large-scale LLM Training — <https://lilianweng.github.io/posts/2021-09-25-train-large/>

10.6 8.6 — Expert Parallelism at Scale

Level: **Expert** **Hook:** “With 256 experts across 64 GPUs, every token routed across the network is a communication event. Here’s how DeepSeek managed it.”

Outline

1. Expert placement: 4 experts per GPU (256 experts, 64 GPUs)
2. The all-to-all communication pattern: tokens dispatched to expert GPUs, then returned

3. Two all-to-all ops per MoE layer: dispatch + combine
4. Limiting factors: network bandwidth, latency, cross-node vs intra-node
5. DeepSeek’s constraint: each token routes to max 4 nodes (to control communication)
6. The intra-node vs inter-node efficiency split

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3.2.3) — [arXiv:2412.19437](#)
- **Paper:** Insights into DeepSeek-V3: Scaling Challenges — [arXiv:2505.09343](#)

10.7 8.7 — Communication Bottlenecks in LLM Training

Level: **Expert** **Hook:** “On 2048 GPUs, the network is the bottleneck — not the GPUs. DeepSeek redesigned the network topology.”

Outline

1. Standard data centre topology: 3-layer Fat-Tree
2. The problem: MoE all-to-all traffic congests the aggregation layer
3. DeepSeek’s solution: Multi-Plane 2-layer Fat-Tree topology
4. Why 2-layer is sufficient when traffic is between fixed expert locations
5. Cost saving: 2-layer vs 3-layer in infrastructure cost
6. IB (InfiniBand) vs NVLink: when to use which
7. Impact on training throughput

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3.4) — [arXiv:2412.19437](#)
- **Paper:** Insights into DeepSeek-V3 (network section) — [arXiv:2505.09343](#)

10.8 8.8 — GPU Memory Optimization Strategies

Level: **Advanced** **Hook:** “Training DeepSeek-V3 required fitting 671B parameters in GPU memory. Here’s every trick used.”

Outline

1. Activation checkpointing (gradient checkpointing): trade compute for memory
2. Recomputation strategy: only checkpoint expensive layers (MoE dispatch)
3. FP8 parameter storage: 1 byte vs 2 (BF16) per weight element
4. Optimizer state: Adam stores 3 copies — how to reduce
5. ZeRO sharding (DeepSpeed): partition optimizer states across data-parallel ranks
6. CPU offloading: move less-used tensors to RAM
7. Memory budget breakdown for DeepSeek-V3 training

Resources

- **Paper:** ZeRO: Memory Optimizations Toward Training Trillion Parameter Models — [arXiv:1910.02054](#)
- **Blog:** Hugging Face — Model Memory Anatomy — https://huggingface.co/docs/transformers/model_memory_anatomy
- **Docs:** DeepSpeed — <https://www.deepspeed.ai/>

11. Phase 9 — DeepSeek R1 & Reasoning

Phase 9 — DeepSeek R1 & Reasoning (6 Articles)

Goal: DeepSeek-R1 is the most impactful release in the series. This phase covers the full technical story: RL for reasoning, the GRPO algorithm, emergent CoT, and test-time compute scaling.

11.1 9.1 — DeepSeek R1 Architecture

Level: **Advanced** **Hook:** “R1 is not a new architecture. It is DeepSeek-V3 trained differently. The difference is everything.”

Outline

1. R1-Zero: V3-Base + pure RL (no SFT), GRPO algorithm
2. R1: 4-stage pipeline (cold start → RL → rejection sampling SFT → final RL)
3. Distilled models: 1.5B, 7B, 8B, 14B, 32B, 70B (Qwen and LLaMA bases)
4. The `<think>` / `<answer>` format: how reasoning is structured in the output
5. Context window: 128K (same as V3)
6. Inference: longer reasoning chain = better answers (test-time compute)

Resources

- **Primary:** DeepSeek-R1 Paper — [arXiv:2501.12948](https://arxiv.org/abs/2501.12948)
- **Paper (Nature):** DeepSeek-R1 peer-reviewed — <https://www.nature.com/articles/s41586-025-09422-z>

11.2 9.2 — Reinforcement Learning for Reasoning Models

Level: **Advanced** **Hook:** “RLHF trains models to be helpful. RLVR trains them to be *correct*. Here’s the difference.”

Outline

1. RLHF recap: human preference reward model, PPO optimisation
2. The problem with RLHF for reasoning: humans can’t evaluate complex maths
3. RLVR (Verifiable Rewards): reward is objectively computed — math answer correct/wrong, code passes tests
4. Why verifiable rewards enable self-improvement without human annotation
5. The reward model in R1: accuracy reward + format reward (no “model-based” reward)
6. Why rule-based rewards avoid reward hacking

Resources

- **Paper:** DeepSeek-R1 (Section 2) — [arXiv:2501.12948](https://arxiv.org/abs/2501.12948)
- **Paper:** InstructGPT (RLHF) — [arXiv:2203.02155](https://arxiv.org/abs/2203.02155)
- **Blog:** Vellum — How DeepSeek-R1 Was Built — <https://www.vellum.ai/blog/the-training-of-deepseek>

11.3 9.3 — GRPO vs PPO Explained

Level: **Advanced** **Hook:** “PPO needs a critic model as large as the policy. GRPO threw out the critic and trained faster. Here’s the math.”

Outline

1. PPO recap: policy + value network, clipped surrogate objective, GAE
2. The critic’s cost: same size as the policy model → 2× memory and compute

3. GRPO (Group Relative Policy Optimization):
 - Sample G outputs per prompt: $\{o_1, o_2, \dots, o_G\}$
 - Compute reward for each: $\{r_1, r_2, \dots, r_G\}$
 - Advantage: $\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r}$ (normalise within the group)
 - No value network needed: baseline estimated from group mean
4. GRPO objective: clip + KL divergence from reference model
5. Why group-relative advantage is sufficient for reasoning tasks
6. Comparison table: PPO vs GRPO — memory, speed, quality

Resources

- **Paper:** GRPO original (DeepSeekMath) — [arXiv:2402.03300](#)
- **Paper:** PPO — [arXiv:1707.06347](#)
- **Blog:** Hugging Face TRL GRPO — https://huggingface.co/docs/trl/grpo_trainer
- **Blog:** HuggingFace R1 explained — <https://huggingface.co/blog/NormalUhr/deepseek-r1-explained>

11.4 9.4 — Emergent Chain-of-Thought Reasoning

Level: **Advanced** **Hook:** “No one showed R1-Zero how to think step by step. It figured it out by itself, after 600 RL steps.”

Outline

1. What is chain-of-thought (CoT)? Manual prompting vs automatic emergence
2. R1-Zero training: only told “produce a thinking process inside `<think>` tags”
3. The “aha moment”: at 600 training steps, the model starts generating multi-step reasoning
4. Emergent behaviours: self-verification, backtracking, reflection
5. AIME 2024 trajectory: pass@1 goes 15.6% → 71.0% during RL training
6. Why this emergence is surprising: the reward signal says nothing about *how* to reason
7. Comparison to GPT-o1: similar emergent reasoning, different training recipe

Resources

- **Paper:** DeepSeek-R1 (Section 3 — R1-Zero analysis) — [arXiv:2501.12948](#)
- **Paper:** “Chain-of-Thought Prompting Elicits Reasoning” — [arXiv:2201.11903](#)
- **Blog:** Medium — DeepSeek-R1: Redefining Reasoning — Sushant Gautam

11.5 9.5 — Self-Verification in Reasoning Models

Level: **Expert** **Hook:** “The smartest thing R1 does: check its own work. Here’s what self-verification actually looks like in the activations.”

Outline

1. What self-verification looks like in R1 outputs: “wait, let me recalculate...”
2. Is this genuine metacognition or a learned pattern?
3. Process Reward Models (PRMs) as an explicit self-verification signal
4. Outcome Reward Models (ORMs) vs PRMs for training
5. Why PRMs are expensive to train but produce better reasoning
6. DeepSeek-V3.2’s RLVR: “correct answers don’t guarantee correct reasoning”
7. Research frontier: learning to verify as a skill separate from solving

Resources

- **Paper:** “Let’s Verify Step by Step” (Process Reward Models) — [arXiv:2305.20050](#)
- **Paper:** DeepSeek-V3.2 — [arXiv:2512.02556](#)

- **Paper:** “Scaling LLM Test-Time Compute” — [arXiv:2408.03314](#)

11.6 9.6 — Test-Time Compute Scaling

Level: **Expert** **Hook:** “Training-time compute is limited by money. Test-time compute is limited by patience. R1 showed which matters more.”

Outline

1. The test-time compute scaling hypothesis (OpenAI o1 introduced this publicly)
2. Two strategies: longer chains of thought vs majority voting (best-of- N)
3. Scaling curve: for reasoning tasks, longer thinking = better answers up to a limit
4. R1’s context window and extended thinking: 128K tokens of reasoning
5. Comparison: \$1 in training compute vs \$1 in inference compute — which buys more quality?
6. The hardware economics: inference chips vs training chips
7. Future: can test-time scaling replace larger models?

Resources

- **Paper:** “Scaling LLM Test-Time Compute Optimally” — [arXiv:2408.03314](#)
- **Paper:** “Let’s Think Step by Step” — [arXiv:2205.01068](#)
- **Blog:** Lilian Weng — Extrinsic Hallucinations in LLMs (test-time methods)

12. Phase 10 — Future DeepSeek Systems

Phase 10 — Future DeepSeek Systems (6 Articles)

Goal: Synthesise the series and look forward. What comes after V4? What does DeepSeek’s engineering philosophy mean for the future of open-source AI? These articles are more speculative but grounded in the technical trends of the series.

12.1 10.1 — DeepSeek Agentic Architecture

Level: **Advanced** **Hook:** “An LLM that answers questions is impressive. An LLM that uses tools, plans, and acts is transformative.”

Outline

1. What is an agent? The perceive-plan-act loop
2. Tool use: code interpreter, web search, file I/O
3. DeepSeek-R1’s reasoning as an agentic foundation: long CoT = multi-step planning
4. Multi-agent systems: orchestrator + specialist agents
5. Memory: in-context (limited), external (databases), parametric (model weights)
6. Challenges: error propagation, hallucinated tool calls, long-horizon planning
7. Current landscape: OpenAI Operator, Claude Computer Use, DeepSeek’s direction

Resources

- **Blog:** Lilian Weng — LLM Powered Autonomous Agents — <https://lilianweng.github.io/posts/2023-06-23-agent/>
- **Paper:** ReAct: Synergising Reasoning and Acting — [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- **Paper:** Toolformer — [arXiv:2302.04761](https://arxiv.org/abs/2302.04761)

12.2 10.2 — DeepSeek V4 Architecture

Level: **Expert** **Hook:** “1.6T parameters. 1M token context. \$27% of the inference FLOPs of V3.2. How?”

Outline

1. V4 release: April 24, 2026. V4-Base and V4-Pro variants
2. mHC (modified Hyper-Connection): replaces residual connections
 - Constrained hyper-connection scheme for gradient propagation
 - Why deep MoE stacks suffer signal degradation and how mHC fixes it
3. Hybrid attention: combines full attention and sparse attention for 1M context
4. Only 27% of V3.2’s per-token inference FLOPs at 1M context
5. Only 10% of V3.2’s KV cache footprint
6. Coding specialisation: why V4 focuses on software engineering tasks
7. NIST CAISI evaluation results

Resources

- **Primary:** DeepSeek-V4 Technical Report (April 2026) — <https://github.com/deepseek-ai>
- **Blog:** WaveSpeed V4 analysis — <https://wavespeed.ai/blog/posts/deepseek-v4-everything-we-know-a>
- **Background:** Hyper-Connections paper — search arXiv for “hyper-connection transformer”

12.3 10.3 — Million-Token Context Windows

Level: **Advanced** **Hook:** “A 1M token context window can hold 50 novels. The engineering to make it work is the real story.”

Outline

1. Why 1M context: entire codebases, long documents, multi-session memory
2. The two bottlenecks: attention compute $O(n^2)$ and KV cache $O(n)$
3. Solving $O(n^2)$: sparse attention, sliding window, hybrid full+sparse
4. Solving $O(n)$ KV cache: MLA (Phase 3) + mHC efficiency (V4)
5. YaRN and LongRoPE: extending RoPE to long contexts
6. Needle-in-a-haystack evaluations: does the model actually use 1M tokens?
7. Hardware requirements: V4 at 1M context — how many GPUs, how much VRAM?

Resources

- **Paper:** Ring Attention — [arXiv:2310.01889](#)
- **Paper:** LongRoPE — [arXiv:2402.13753](#)
- **Paper:** YaRN — [arXiv:2309.00071](#)
- **Paper:** “Lost in the Middle” (long context failures) — [arXiv:2307.03172](#)

12.4 10.4 — AI Infrastructure Beyond CUDA

Level: **Expert** **Hook:** “NVIDIA’s H800 is what DeepSeek had. DeepSeek’s technical report suggests what hardware *should* exist.”

Outline

1. DeepSeek’s hardware wishlist (from V3 Technical Report Section 3.5):
 - Tile-level FP8 accumulation natively in hardware
 - Scale-up/scale-out convergence (NVLink ranges becoming network ranges)
 - Sub-microsecond latency interconnects
2. The CUDA monopoly: why alternative chips (AMD, Groq, Tenstorrent, Cerebras) exist
3. Chinese hardware constraints: H800 vs A100 vs domestic Huawei Ascend
4. Custom ASICs for inference (speculative decoding, MoE routing)
5. The open question: can hardware innovation outpace model innovation?

Resources

- **Primary:** DeepSeek-V3 Technical Report (Section 3.5 — Future Hardware Directions) — [arXiv:2412.19437](#)
- **Paper:** Insights into DeepSeek-V3 (hardware discussion) — [arXiv:2505.09343](#)

12.5 10.5 — DeepSeek vs GPT-5 vs Gemini

Level: **Intermediate** **Hook:** “Open weights vs closed weights. The battle for AI’s future is not about benchmarks. It’s about who controls the infrastructure.”

Outline

1. Objective comparison: benchmarks where each model leads (as of mid-2026)
2. The strategic difference: DeepSeek open-weights; OpenAI/Google closed
3. Economic comparison: running DeepSeek-V4 vs GPT-5 API at scale
4. Capability gaps that remain: multimodal, real-time, agentic robustness
5. The ecosystem argument: open models drive research, tooling, fine-tuning
6. The risk argument: open frontier models and safety
7. Author’s balanced assessment (present multiple views)

12.6 10.6 — Future of Open-Source Frontier Models

Level: **Intermediate** **Hook:** “Three years ago, GPT-3 was untouchable. Today, DeepSeek-R1 is free to download. What does 2027 look like?”

Outline

1. The open-source convergence: how open models close the quality gap over time
2. Hardware democratisation: when will consumer hardware run frontier models?
3. The alignment gap: open weights + unknown training = safety uncertainty
4. DeepSeek’s role as a force multiplier for the research community
5. Prediction: 3 technical trends that will define the next generation
6. A call to action for readers: how to contribute to this ecosystem
7. Series wrap-up: the 10-phase journey from attention to frontier models

Resources

- **Blog:** Hugging Face Open LLM Leaderboard — https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard
- **Paper:** “DeepSeek: Paradigm Shifts and Technical Evolution” — [arXiv:2507.09955](https://arxiv.org/abs/2507.09955)

13. Master Resource Library

13.1 Official DeepSeek Papers (Complete List)

Paper Title	arXiv / Source
DeepSeekMoE: Towards Ultimate Expert Specialization	arXiv:2401.06066
DeepSeek-Coder: When LLM Meets Programming	arXiv:2401.14196
DeepSeekMath: Pushing Limits of Mathematical Reasoning	arXiv:2402.03300
DeepSeek-V2: Strong, Economical, Efficient MoE LM	arXiv:2405.04434
DeepSeek-Coder-V2: Breaking Barrier of Closed-Source	arXiv:2406.11931
DeepSeek-V3 Technical Report	arXiv:2412.19437
DeepSeek-R1: Incentivizing Reasoning via RL	arXiv:2501.12948
DeepSeek-V3.2 Technical Report	arXiv:2512.02556
DeepSeek-V4 Technical Report	github.com/deepseek-ai (2026)
Insights into DeepSeek-V3: Scaling Challenges & Hardware	arXiv:2505.09343

13.2 Foundational Papers (Read Before Writing Each Phase)

Phase	Paper	arXiv
1	Attention Is All You Need	arXiv:1706.03762
1	Language Models are Few-Shot Learners (GPT-3)	arXiv:2005.14165
2	Fast Transformer Decoding: MQA (Shazeer)	arXiv:1911.02150
2	GQA: Training Generalised MQT Models	arXiv:2305.13245
2	FlashAttention-2	arXiv:2307.08691
3	TransMLA: MLA Is All You Need	arXiv:2502.07864
3	Towards Economical Inference (MHA2MLA)	arXiv:2502.14837
4	RoFormer: RoPE	arXiv:2104.09864
4	YaRN: Efficient Context Window Extension	arXiv:2309.00071
5	Sparsely-Gated MoE (Shazeer 2017)	arXiv:1701.06538
5	Switch Transformers	arXiv:2101.03961
5	Mixtral of Experts	arXiv:2401.04088
6	Multi-Token Prediction (Meta)	arXiv:2404.19737
6	Speculative Decoding	arXiv:2211.17192
7	Mixed Precision Training	arXiv:1710.03740
7	FP8 Formats for Deep Learning	arXiv:2209.05433
7	GPTQ Post-Training Quantisation	arXiv:2210.17323
8	Megatron-LM	arXiv:1909.08053
8	ZeRO: Memory Optimizations	arXiv:1910.02054
9	PPO (Schulman et al.)	arXiv:1707.06347
9	InstructGPT (RLHF)	arXiv:2203.02155
9	Let's Verify Step by Step (PRMs)	arXiv:2305.20050
9	Chain-of-Thought Prompting	arXiv:2201.11903
10	Scaling LLM Test-Time Compute	arXiv:2408.03314
10	ReAct: Reasoning + Acting	arXiv:2210.03629

13.3 Essential Blogs and Online Resources

- **Jay Alammar** — The Illustrated Transformer, Illustrated GPT-2
<https://jalammar.github.io>
- **Sebastian Raschka** — LLMs from Scratch (book + GitHub + blog)
<https://sebastianraschka.com> · <https://magazine.sebastianraschka.com>
- **Lilian Weng** — Comprehensive ML blog (attention, agents, RLHF, inference)
<https://lilianweng.github.io>
- **Lena Voita** — NLP Course with beautiful visualisations
https://lena-voita.github.io/nlp_course
- **Andrej Karpathy** — Zero-to-Hero video series (tokenizer, GPT, nanoGPT)

<https://karpathy.ai>

- **Hugging Face Blog** — MoE, quantisation, speculative decoding, KV cache
<https://huggingface.co/blog>
- **The Annotated Transformer** — Harvard NLP
<https://nlp.seas.harvard.edu/annotated-transformer/>
- **Eleuther AI Blog** — RoPE, evaluation, scaling laws
<https://blog.eleuther.ai>
- **WaveSpeed Blog** — DeepSeek V4 analysis
<https://wavespeed.ai/blog>

13.4 Code Repositories

- **DeepSeek Official:** <https://github.com/deepseek-ai>
- **nanoGPT (Karpathy):** <https://github.com/karpathy/nanoGPT>
- **LLMs-from-Scratch (Raschka):** <https://github.com/rasbt/LLMs-from-scratch>
- **HuggingFace Transformers:** <https://github.com/huggingface/transformers>
- **vLLM (PagedAttention):** <https://github.com/vllm-project/vllm>
- **DeepSpeed (ZeRO):** <https://github.com/microsoft/DeepSpeed>
- **TRL (GRPO trainer):** <https://github.com/huggingface/trl>
- **BertViz (attention visualiser):** <https://github.com/jessevig/bertviz>
- **NVIDIA TransformerEngine (FP8):** <https://github.com/NVIDIA/TransformerEngine>

14. Writing Guide & Article Template

14.1 The Standard Article Structure (LaTeX Template)

Every article in the series follows this exact $\text{\LaTeX}$ skeleton:

Listing 1: Reusable Article Template

```

1 \documentclass[12pt,a4paper]{article}
2 % [shared preamble      same packages as this roadmap]
3
4 \begin{document}
5
6 % === HEADER ===
7 % Phase X      Article Y.Z      Difficulty: [Beginner/Intermediate/Advanced/Expert]
8 % Series: DeepSeek Engineering Blog Series
9
10 % === SECTION 1: THE HOOK ===
11 % 1-2 sentences. No jargon. Should work as a tweet.
12 \section*{The Hook}
13
14 % === SECTION 2: FOR EVERYONE ===
15 % The Big Idea + Simple Analogy + Why It Matters callout box
16 \section{For Everyone: The Big Idea}
17 \begin{greenbox}[If You Read Nothing Else...]
18 [The single most important insight. 3 sentences max.]
19 \end{greenbox}
20
21 % === SECTION 3: FOR ENGINEERS ===
22 % System design, architecture, trade-offs. No equations yet.
23 \section{For Engineers: How It Works}
24
25 % === SECTION 4: FOR RESEARCHERS ===
26 % Full equations, pseudocode, ablations, citations.
27 \section{Deep Dive: The Full Technical Story}
28
29 % === SECTION 5: KEY TAKEAWAYS ===
30 \section{5 Things to Remember}
31 \begin{enumerate}
32 \item \textbf{[Key word]:} ...
33 \item \textbf{[Key word]:} ...
34 \item \textbf{[Key word]:} ...
35 \item \textbf{[Key word]:} ...
36 \item \textbf{[Key word]:} ...
37 \end{enumerate}
38
39 % === SECTION 6: FURTHER READING ===
40 \section{Go Deeper}
41
42 % === SECTION 7: WHAT'S NEXT ===
43 \section*{Next in the Series}
44 % 1 paragraph. Bridge to next article. Create desire.
45
46 \end{document}

```

14.2 Tone, Voice, and Style Rules

Never break these rules: (1) No sentence longer than 25 words in Sections 1 and 2. (2) Never introduce an acronym without defining it, even if it was defined in a previous article. (3) Every claim about performance must cite a specific number and source. (4) Every analogy must be tested: “Would my parent understand this?”

- **Active voice always.** “DeepSeek compressed the KV cache” not “The KV cache was compressed.”
- **Concrete numbers always.** “671 billion parameters,” not “a very large number of parameters.”
- **Show, don’t tell.** Never use “revolutionary,” “groundbreaking,” or “unprecedented.” Show the numbers instead.
- **One idea per paragraph.** Maximum 4 sentences per paragraph in non-technical sections.
- **Analogy before equation.** Every technical concept gets a plain-English analogy before its mathematical formulation.
- **Consistent terminology.** Use the exact same term for each concept throughout. No synonyms for technical terms.

14.3 The Publishing Checklist

Check every item before hitting publish.

- ☐ A non-technical person can understand the Hook and Big Idea sections
- ☐ Every acronym is defined on first use in this article
- ☐ Every performance claim has a citation or data source
- ☐ Every equation has a plain-English interpretation immediately after it
- ☐ The article has at least one diagram or table
- ☐ The article compiles to PDF without errors (`pdflatex` run twice)
- ☐ The PDF is proof-read (not the `.tex` source)
- ☐ A bridge sentence to the next article appears at the end
- ☐ The article is cross-posted to Medium/dev.to/Substack
- ☐ The `.tex` source is committed to the GitHub repository